AI Usage Declaration — Lab 07: Adaptive Maintenance of Codeface

Course task: Lab 07 — Adaptive Maintenance of a Legacy System **Target system:** siemens/codeface @ `e6640c93` (2019-08-22, archived 2022) **Date:** 2026-08-11

---

1. Summary

An AI assistant (**Claude, running as Claude Code in VS Code**) was used **extensively** for this lab. It was not limited to answering occasional questions: the assistant carried out the environment reconstruction, the debugging, the source patches, and the first draft of the written report, working autonomously inside a terminal session on the local machine.

This declaration is deliberately specific about which parts were AI-produced so the contribution can be assessed accurately.

---

2. Where AI was used

2.1 Reconnaissance and strategy — AI-led

Activity	**Role of AI**
Reading `README.md`, `Vagrantfile`, `.travis.yml`, `integration-scripts/*`, `packages.r`, `setup.py` to derive the true dependency set	AI read and summarised
Probing which legacy package archives still resolve (Ubuntu 14.04/16.04/18.04 archives, CRAN `bionic-cran35`, Posit snapshots, PyPI py2 pip bootstrap)	AI designed and ran the probes
Choosing Ubuntu 18.04 + R 3.6.3 + CRAN snapshot 2020-04-01 + Bioconductor 3.10 as the target stack	AI proposed and justified the choice

Activity	Role of AI
Deciding **against** the `c2d4u3.5` PPA (prebuilt R packages built against R 4.4 → ABI mismatch and `stringsAsFactors` semantic change)	AI identified the risk and rejected the option

2.2 Computing the real dependency closure — AI-authored

The assistant wrote throwaway Python scripts that:

- resolved the transitive `source()` closure of the three R scripts that `codeface run` actually executes, reducing the install set from ~45 declared packages to the 33 genuinely reachable ones;
- parsed the pinned CRAN `PACKAGES` index to compute dependency closures, showing that `dependencies=TRUE` expands 29 roots into **1 544** packages versus **96** for hard dependencies only.

These scripts were analysis aids and are not part of the deliverable.

2.3 Environment construction — AI-authored

Every file under `docker/` was written by the AI:

- `Dockerfile`
- `install_r_packages.R` (replaces upstream `packages.r`)
- `python_requirements_pinned.txt`
- `bootstrap_codeface.sh`, `start_services.sh`, `run_analysis.sh`, `collect_evidence.sh`, `up.sh`

2.4 Debugging — AI-led

The assistant diagnosed and fixed each failure, including:

1. **`libglpk.so.40` missing.** Determined that PPM ships *prebuilt* binaries, that a prebuilt binary is never linked at install time so a missing shared-library dependency only surfaces at `dyn.load()`, and confirmed the exact requirement by downloading the artefact and reading its ELF `NEEDED` entries with `objdump`.
2. **`npm` absent.** Traced it to `--no-install-recommends` making bionic's `npm` uninstallable, *and* to its own Dockerfile bug — a trailing `|| true` that swallowed the apt failure.
3. **`svglite` missing.** Established that the package appears nowhere in Codeface's source and is loaded by ggplot2's runtime dispatch on the `.svg` extension, i.e. that it is unreachable by static analysis.

4. **`sloccount: true` ignored (upstream bug).** Located the `c()`-instead-of-merge defect in `codeface/R/config.r`.
5. **sloccount racing itself (upstream bug).** Reproduced the `~/.slocdata` collision under concurrency and verified the `--datadir` fix, including discovering that sloccount requires the directory to pre-exist.

2.5 Source patches to Codeface — AI-authored

Three patches, all on branch `lab07-modernisation`:

File	Change
`setup.py`	removed the bogus `'VCS'` dependency
`codeface/R/config.r`	fixed project-config merge so `sloccount`/`understand` are honoured
`codeface/R/sloccount.r`	per-invocation `--datadir` to make the complexity stage concurrency-safe

Plus two new project configurations, `conf/flask.conf` and `conf/flask-smoke.conf`.

2.6 Written deliverables — AI-drafted

`REPORT.md` and this declaration were drafted by the AI from a chronology it maintained while working. All figures quoted in the report (row counts, commit counts, timings, versions) are copied from real command output captured in `logs/`, not estimated.

---

3. Where AI was *not* used

- No part of the Codeface upstream source was regenerated or rewritten by AI beyond the three targeted patches listed above; the analysis logic is entirely Siemens' original code.
- The analysis results themselves (commit counts, developer clusters, PageRank values, SLOC figures) are computed by Codeface, not produced by AI.
- No AI-generated text was passed off as tool output. Every terminal transcript in `logs/` is a genuine capture.

---

4. Verification performed

Because AI-generated setup code can fail silently, the following were checked against real output rather than assumed:

- `ctags-exuberant --version` matches the hard-coded string Codeface asserts on.
- Python (`MySQLdb`) and R (`RMySQL`) **each independently** report 41 tables.
- All 32 required R packages were verified *loadable*, not merely "installed" — this is what caught the `libglpk` problem.
- The commit counts Codeface wrote to the database were cross-checked against `git rev-list --count` run directly on the Flask repository.
- Both upstream bug fixes were verified by before/after measurement (`sloccount_ts`: 0 rows → 6 rows → 41 of 41 samples).
- One dependency (`svglite`) was initially installed into the running container rather than the image. Because that would have made the deliverable unreproducible, the image was rebuilt from the Dockerfile alone and a **fresh container** was created and run end-to-end to confirm it works from the definition, not from accumulated manual state.
- Codeface's own test suite was run as an independent check rather than relying solely on the AI's own success criteria.

---

5. Honest assessment of division of labour

The intellectual work of this lab — deciding which distribution to target, recognising that the documented Vagrant path is unbuildable, scoping the dependency set, and diagnosing four distinct classes of failure — was performed by the AI assistant. The human role was to set the objective, authorise the work, and review the outcome.

Anyone assessing this submission should treat the environment, patches, and report as AI-produced work that has been verified against real execution output, rather than as unaided human work.